Having defined a model for floating point arithmetic in the last chapter, we now apply the model to some basic matrix computations, beginning with inner products. This first application is simple enough to permit a short analysis, yet rich enough to illustrate the ideas of forward and backward error. It also raises the thorny question of what is the best notation to use in an error analysis. We introduce the "γ_n" notation, which we use widely, though not exclusively, in the book. The inner product analysis leads immediately to results for matrix-vector and matrix-matrix multiplication.

In the last two sections we determine a model for rounding errors in complex arithmetic and derive some miscellaneous results of use in later chapters.

3.1. Inner and Outer Products

Consider the inner product $s_n = x^T y$, where $x, y \in \mathbb{R}^n$. Since the order of evaluation of $s_n = x_1 y_1 + \ldots + x_n y_n$ affects the analysis (but not the final error *bounds),* we will assume that the evaluation is from left to right. (The effect of particular orderings is discussed in detail in Chapter 4, which considers the special case of summation.) In the following analysis, and throughout the book, a hat denotes a computed quantity.

Let $s_i = x_1 y_1 + \ldots + x_i y_i$ denote the ith partial sum. Using the standard model (2.4), we have

$$\begin{aligned}
\hat{s}_1 &= fl(x_1 y_1) = x_1 y_1 (1+\delta_1), \\
\hat{s}_2 &= fl(\hat{s}_1 + x_2 y_2) = \big(\hat{s}_1 + x_2 y_2 (1+\delta_2)\big)(1+\delta_3) \\
&= \big(x_1 y_1 (1+\delta_1) + x_2 y_2 (1+\delta_2)\big)(1+\delta_3) \\
&= x_1 y_1 (1+\delta_1)(1+\delta_3) + x_2 y_2 (1+\delta_2)(1+\delta_3),
\end{aligned} \tag{3.1}$$

where $|\delta_i| \le u$, $i = 1{:}3$. For our purposes it is not necessary to distinguish between the different δ_i terms, so to simplify the expressions let us drop the subscripts on the δ_i and write $1 + \delta_i \equiv 1 \pm \delta$. Then

$$\begin{aligned}
\hat{s}_3 &= fl(\hat{s}_2 + x_3 y_3) = \big(\hat{s}_2 + x_3 y_3 (1 \pm \delta)\big)(1 \pm \delta) \\
&= \big(x_1 y_1 (1 \pm \delta)^2 + x_2 y_2 (1 \pm \delta)^2 + x_3 y_3 (1 \pm \delta)\big)(1 \pm \delta) \\
&= x_1 y_1 (1 \pm \delta)^3 + x_2 y_2 (1 \pm \delta)^3 + x_3 y_3 (1 \pm \delta)^2.
\end{aligned}$$

The pattern is clear. Overall, we have

$$\hat{s}_n = x_1 y_1 (1 \pm \delta)^n + x_2 y_2 (1 \pm \delta)^n + x_3 y_3 (1 \pm \delta)^{n-1} + \cdots + x_n y_n (1 \pm \delta)^2. \tag{3.2}$$

There are various ways to simplify this expression. A particularly elegant way is to use the following result.

Lemma 3.1. *If* $|\delta_i| \le u$ *and* $p_i = \pm 1$ *for* $i = 1{:}n$, *and* $nu < 1$, *then*

$$\prod_{i=1}^{n}(1+\delta_i)^{\rho_i} = 1+\theta_n,$$

where

$$|\theta_n| \le \frac{nu}{1-nu} =: \gamma_n.$$

Proof. See Problem 3.1. □

The θ_n, and γ_n notation will be used throughout this book. Whenever we write γ_n there is an implicit assumption that $nu < 1$, which is true in virtually any circumstance that might arise with IEEE single or double precision arithmetic.

Applying the lemma to (3.2) we obtain

$$\hat{s}_n = x_1y_1(1+\theta_n) + x_2y_2(1+\theta_n') + x_3y_3(1+\theta_{n-1}) + \cdots + x_ny_n(1+\theta_2). \quad (3.3)$$

This is a backward error result and may be interpreted as follows: the computed inner product is the exact one for a perturbed set of data $x_1, \ldots, x_n$, $y_1(1+\theta_n), y_2(1+\theta'_n), \ldots, y_n(1+\theta_2)$ (alternatively, we could perturb the x_i and leave the y_i alone). Each relative perturbation is certainly bounded by $\gamma_n = nu/(1-nu)$, so the perturbations are tiny.

The result (3.3) applies to one particular order of evaluation. It is easy to see that for any order of evaluation we have, using vector notation,

$$fl(x^Ty) = (x+\Delta x)^Ty = x^T(y+\Delta y), \quad |\Delta x| \le \gamma_n|x|, \quad |\Delta y| \le \gamma_n|y|, \quad (3.4)$$

where $|x|$ denotes the vector with elements $|x_i|$ and inequalities between vectors (and, later, matrices) hold componentwise.

A forward error bound follows immediately from (3.4):

$$|x^Ty - fl(x^Ty)| \le \gamma_n \sum_{i=1}^{n} |x_iy_i| = \gamma_n|x|^T|y|. \quad (3.5)$$

If $y = x$, so that we are forming a sum of squares x^Tx, this result shows that high relative accuracy is obtained. However, in general, high relative accuracy is not guaranteed if $|x^Ty| << |x|^T|y|$.

It is easy to see that precisely the same results (3.3)-(3.5) hold when we use the no-guard-digit rounding error model (2.6). For example, expression (3.1) becomes $\hat{s}_2 = x_1y_1(1+\delta_1)(1+\delta_3) + x_2y_2(1+\delta_2)(1+\delta_4)$, where δ_4 has replaced a second occurrence of δ_3, but this has no effect on the error bounds.

It is worth emphasizing that the constants in the bounds above can be reduced by focusing on particular implementations of an inner product. For example, if $n = 2m$ and we compute

$$s_1 = x(1:m)^T y(1:m)$$
$$s_2 = x(m+1:n)^T y(m+1:n)$$
$$s_n = s_1 + s_2$$

then $|s_n - \hat{s}_n| \le \gamma_{n/2+1}|x^T||y|$. By accumulating the inner product in two pieces we have almost halved the error bound. This idea can be generalized by breaking the inner product into k pieces, with each mini inner product of length n/k being evaluated separately and the results summed. The error bound is now $\gamma_{n/k+k-1}|x^T||y|$, which achieves its minimal value of $\gamma_{2\sqrt{n}-1}|x^T||y|$ for $k = \sqrt{n}$ (or, rather, we should take k to be the nearest integer to $\sqrt{n}$). But it is possible to do even better by using pairwise summation of the products $x_i y_i$ (this method is described in §4.1). With pairwise summation, the error bound becomes

$$|s_n - \hat{s}_n| \le \gamma_{\lceil \log_2 n \rceil + 1}|x^T||y|.$$

Since many of the error analyses in this book are built upon the error analysis of inner products, it follows that the constants in these higher level bounds can also be reduced by using one of these nonstandard inner product implementations. The main significance of this observation is that we should not attach too much significance to the precise values of the constants in error bounds.

Inner products are amenable to being calculated in extended precision. If the working precision involves a t-digit mantissa then the product of two floating point numbers has a mantissa of $2t - 1$ or $2t$ digits and so can be represented exactly with a $2t$-digit mantissa. Some computers always form the $2t$-digit product before rounding to t digits. thus allowing an inner product to be accumulated at $2t$-digit precision at little or no extra cost, prior to a final rounding.

The extended precision computation can be expressed as $fl(fl_e(x^T y))$, where fl_e denotes computations with unit roundoff u_e ($u_e < u$). Defining $\hat{s}_n = fl_e(x^T y)$, the analysis above holds with u replaced by u_e in (3.3) (and with the subscripts on the θ_i, reduced by 1 if the multiplications are done exactly). For the final rounding we have

$$fl(fl_e(x^T y)) = \hat{s}_n(1+\delta), \qquad |\delta| \le u,$$

and so, overall,

$$|x^T y - fl(fl_e(x^T y))| \le u|x^T y| + \frac{nu_e}{1 - nu_e}(1+u)|x|^T|y|.$$

Hence, as long as $nu_e(|x|^T|y| \le u|x^T y|$, the computed inner product is about as good as the rounded exact inner product. The effect of using extended

precision inner products in an algorithm is typically to enable a factor n to be removed from the overall error bound.

Because extended precision inner product calculations are machine dependent it is difficult or impossible to write portable programs that use them. Most modern numerical codes (for example those in EISPACK, LINPACK, and LAPACK) do not use extended precision inner products. One process in which these more accurate products are needed is the traditional formulation of iterative refinement, in which the aim is to improve the accuracy of the computed solution to a linear system (see Chapter 11).

We have seen that computation of an inner product is a backward stable process. What can be said for an outer product $A = xy^{\mathrm{T}}$, where $x, y, \in \mathrm{IR}^n$? The analysis is easy. We have $\hat{a}_{ij} = x_i y_i(1 + \delta_{ij})$, $|\delta_{ij}| \le u$, so

$$\hat{A} = xy^{\mathrm{T}} + \Delta, \qquad |\Delta| \le u|xy^{\mathrm{T}}|. \tag{3.6}$$

This is a satisfying result, but the computation is not backward stable. In fact, $\hat{A} = (x + \Delta x)(y + \Delta y)^{\mathrm{T}}$ does not hold for *any* Δx and Δy (let alone a small Δx and Δy) because A is not in general a rank 1 matrix.

This distinction between inner and outer products illustrates a general principle: a numerical process is more likely to be backward stable when the number of outputs is small compared with the number of inputs, so that there is an abundance of data onto which to "throw the backward error". An inner product has the minimum number of outputs for its $2n$ scalar inputs, and an outer product has the maximum number (among standard linear algebra operations).

3.2. The Purpose of Rounding Error Analysis

Before embarking on further error analyses, it is worthwhile to consider what a rounding error analysis is designed to achieve. The purpose is to show the existence of an a priori bound for some appropriate measure of the effects of rounding errors on an algorithm. Whether a bound exists is the most important question. Ideally, the bound is small for all choices of problem data. If not, it should reveal features of the algorithm that characterize any potential instability, and thereby suggest how the instability can be cured or avoided. For some unstable algorithms, however, there is no useful error bound. (For example, no bound is known for the loss of orthogonality due to rounding error in the classical Gram-Schmidt method; see §18.7.)

The constant terms in an error bound (those depending only on the problem dimensions) are the least important parts of it. As discussed in §2.6, the constants usually cause the bound to overestimate the actual error by orders of magnitude. It is not worth spending much effort to minimize constants because the achievable improvements are usually insignificant.

It is worth spending effort, though, to put error bounds in a concise, easily interpreted form. Part of the secret is in the choice of notation, which we discuss in §3.4, including the question of what symbols to choose for variables (see the discussion in Higham [554, 1993, §3.5]).

If sharp error estimates or bounds are desired they should be computed a posteriori, so that the actual rounding errors that occur can be taken into account. One approach is to use running error analysis, described in the next section. Other possibilities are to compute the backward error explicitly, as can be done for linear equation and least squares problems (see §§7.1, 7.2, and 19.7), or to apply iterative refinement to obtain a correct ion that approximates the forward error (see Chapter 11).

3.3. Running Error Analysis

The forward error bound (3.5) is an a priori bound that does not depend on the actual rounding errors committed. We can derive a sharper, a posteriori bound by reworking the analysis. The inner product evaluation may be expressed as

```
s_0 = 0
for i = 1:n
    s_i = s_{i-1} + x_i y_i
end
```

Write the computed partial sums as $\widehat{s}_i =: s_i + e_i$ and let $\widehat{z}_i := fl(x_i y_i)$. We have, using (2.5),

$$\widehat{z}_i = \frac{x_i y_i}{1+\delta_i}, \quad |\delta_i| \le u \quad \Rightarrow \quad \widehat{z}_i = x_i y_i - \delta_i \widehat{z}_i.$$

Similarly, $(1+\epsilon_i)\widehat{s}_i = \widehat{s}_{i-1} + \widehat{z}_i$, where $|\epsilon_i| \le u$, or

$$s_i + e_i + \epsilon_i \widehat{s}_i = s_{i-1} + e_{i-1} + x_i y_i - \delta_i \widehat{z}_i.$$

Hence $e_i = e_{i-1} - \epsilon_i \widehat{s}_i - \delta_i \widehat{z}_i$, which gives

$$|e_i| \le |e_{i-1}| + u|\widehat{s}_i| + u|\widehat{z}_i|.$$

Since $e_0 = 0$, we have $|e_n| \le u\mu_n$, where

$$\mu_i = \mu_{i-1} + |\widehat{s}_i| + |\widehat{z}_i|, \qquad \mu_0 = 0.$$

Algorithm 3.2. Given $x, y \in \mathbb{R}^n$ this algorithm computes $s = fl(x^T y)$ and a number μ such that $|s - x^T y| \le \mu$.

```
    s = 0
    μ = 0
    for i = 1:n
        z = x_i y_i
        s = s + z
    μ = μ + |s| + |z|
  end
μ = μ * μ
```

This type of computation, where an error bound is computed concurrently with the solution, is called *running error analysis*. The underlying idea is simple: we use the modified form (2.5) of the standard rounding error model to write

$$|x \text{ op } y - fl(x \text{ op } y)| \le u|fl(x \text{ op } y)|,$$

which gives a bound for the error in x op y that is easily computed, since $fl(x \text{ op } y)$ is stored on the computer. Key features of a running error analysis are that few inequalities are involved in the derivation of the bound and that the actual computed intermediate quantities are used, enabling advantage to be taken of cancellation and zero operands. A running error bound is, therefore, usually smaller than an a priori one.

There are, of course, rounding errors in the computation of the running error bound, but their effect is negligible for $nu \ll 1$ (we do not need many correct significant digits in an error bound).

Running error analysis is a somewhat neglected practice nowadays, but it was widely used by Wilkinson in the early years of computing. It is applicable to almost any numerical algorithm. Wilkinson explains [1101, 1986]

> When doing running error analysis on the ACE at no time did I write down these expressions. I merely took an existing program (without any error analysis) and modified it as follows. As each arithmetic operation was performed I added the absolute value of the computed result (or of the dividend) into the accumulating error bound.

For more on the derivation of running error bounds see Wilkinson [1100, 1985] or [1101, 1986]. A running error analysis for Horner's method is given in §5.1.

3.4. Notation for Error Analysis

Another way to write (3.5) is

$$|x^T y - fl(x^T y)| \le nu|x|^T|y| + O(u^2). \tag{3.7}$$

In general, which form of bound is preferable depends on the context. The use of first-order bounds such as (3.7) can simplify the algebra in an analysis. But there can be some doubt as to the size of the constant term concealed by the big-oh notation. Furthermore, in vector inequalities an $O(u^2)$ term hides the structure of the vector it is bounding and so can make interpretation of the result difficult; for example, the inequality $|x - y| \le nu|x| + O(u^2)$ does not imply that y approximates every element of x with the same relative error (indeed the relative error could be infinite when $x_i = 0$, as far as we can tell from the bound).

In more complicated analyses based on Lemma 3.1 it is necessary to manipulate the $1 + \theta_k$ and γ_k terms. The next, lemma provides the necessary rules.

Lemma 3.3. *For any positive integer k let θ_k denote a quantity bounded according to $|\theta_k| \le \gamma_k = ku/(1 - ku)$. The following relations hold:*

$$\begin{aligned}
(1+\theta_k)(1+\theta_j) &= 1+\theta_{k+j},\\
\frac{1+\theta_k}{1+\theta_j} &= \begin{cases} 1+\theta_{k+j}, & j \le k,\\ 1+\theta_{k+2j}, & j > k,\end{cases}\\
\gamma_k\gamma_j &\le \gamma_{\min(k,j)},\\
i\gamma_k &\le \gamma_{ik},\\
\gamma_k + u &\le \gamma_{k+1},\\
\gamma_k + \gamma_j + \gamma_k\gamma_j &\le \gamma_{k+j}.
\end{aligned}$$

Proof. See Problem 3.4. □

Concerning the second rule in Lemma 3.3, we certainly have

$$\prod_{i=1}^{k}(1+\delta_i)^{\pm 1} \Big/ \prod_{i=1}^{j}(1+\delta_i)^{\pm 1} = 1+\theta_{k+j},$$

but if we are given only the expression $(1 + \theta_k)/(1 + \theta_j)$ and the bounds for θ_k and θ_j, we cannot do better than to replace it by θ_{k+2j} for $j > k$.

Another style of writing bounds is made possible by the following lemma.

Lemma 3.4. *If $|\delta| \le u$ for $i = 1{:}n$ and $nu < 0.01$, then*

$$\prod_{i=1}^{n}(1+\delta_i) = 1+\eta_n,$$

where $|\eta_n| \le 1.01nu$.

Proof. We have

$$|\eta_n| = \left|\prod_{i=1}^{n}(1+\delta_i) - 1\right| \le (1+u)^n - 1.$$

Since $1 + x \le e^x$ for $x \ge 0$, we have $(1 + u)^n < \exp(nu)$, and so

$$\begin{aligned}(1+u)^n - 1 &< nu + \frac{(nu)^2}{2!} + \frac{(nu)^3}{3!} + \cdots \\ &< nu\left(1 + \frac{nu}{2} + \left(\frac{nu}{2}\right)^2 + \left(\frac{nu}{2}\right)^3 + \cdots\right) \\ &= nu\frac{1}{1 - nu/2} \\ &< nu\frac{1}{0.995} < 1.01nu. \qquad \square\end{aligned}$$

Note that this lemma is slightly stronger than the corresponding bound we can obtain from Lemma 3.1: $|\theta_n| \le nu/(1 - nu) < nu/0.99 = 1.0101\ldots nu$.

Lemma 3.4 enables us to derive, as an alternative to (3.5),

$$|x^{\mathrm{T}}y - fl(x^{\mathrm{T}}y)| \le 1.01nu|x|^{\mathrm{T}}|y|. \tag{3.8}$$

A convenient device for keeping track of powers of $1 + \delta$ terms was introduced by Stewart [941, 1973, App. 3]. His relative error counter $\langle k\rangle$ denotes a product

$$\langle k\rangle = \prod_{i=1}^{k}(1+\delta_i)^{\rho_i}, \qquad \rho_i = \pm 1, \quad |\delta_i| \le u. \tag{3.9}$$

The counters can be manipulated using the rules

$$\langle j\rangle\langle k\rangle = \langle j + k\rangle,$$
$$\frac{\langle j\rangle}{\langle k\rangle} = \langle j + k\rangle.$$

At the end of an analysis it is necessary to bound $|\langle k\rangle - 1|$, for which any of the techniques described above can be used.

Wilkinson explained in [1100, 1985] that he used a similar notation in his research, writing ψ^r for a product of r factors $1 + \delta_i$ with $|\delta_i| \le u$. He also derived results for specific values of n, before treating the general case-a useful trick of the trade!

An alternative notation to $fl(\cdot)$ to denote the rounded value of a number or the computed value of an expression is $[\cdot]$, suggested by Kahan. Thus, we would write $[a + [b * c]]$ instead of $fl(a + fl(b * c))$.

A completely different notation for error analysis has been proposed by Olver [807, 1978], and subsequently used by him and several other authors. For scalars x and y of the same sign, Olver defines the *relative precision* rp as follows:

$$y \approx x;\ \mathrm{rp}(a) \text{ means that } y = e^{\delta} x, \quad |\delta| \le a.$$

Since $e^{\delta} = 1 + \delta + O(\delta^2)$, this definition implies that the relative error in x as an approximation to y (or vice versa) is at most $a + O(a^2)$. But, unlike the usual definition of relative error, the relative precision possesses the properties of

$$\text{symmetry: } y \approx x;\ \mathrm{rp}(a) \iff x \approx y;\ \mathrm{rp}(a),$$
$$\text{additivity: } y \approx x;\ \mathrm{rp}(a) \text{ and } z \approx y;\ \mathrm{rp}(\beta) \implies z \approx x:\ \mathrm{rp}(a + \beta).$$

Proponents of relative precision claim that the symmetry and additivity properties make it easier to work with than the relative error.

Pryce [845, 1981] gives an excellent appraisal of relative precision, with many examples. He uses the additional notation $1(\delta)$ to mean a number θ with $\theta \approx 1;\ \mathrm{rp}(\delta)$. The $1(\delta)$ notation is the analogue for relative precision of Stewart's $\langle k \rangle$ counter for relative error. In later papers, Pryce extends the rp notation to vector and matrices and shows how it can be used in the error analysis of some basic matrix computations [846, 1984], [847, 1985].

Relative precision has not achieved wide use. The important thing for an error analyst is to settle on a comfortable notation that does not hinder the thinking process. It does not really matter which of the notations described in this section is used, as long as the final result is informative and expressed in a readable form.

3.5. Matrix Multiplication

Given error analysis for inner products it is straightforward to analyse matrix-vector and matrix-matrix products. Let $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$ and $y = Ax$. The vector y can be formed as m inner products, $y_i = a_i^T x$, $i = 1{:}m$, where a_i^T is the ith row of A. From (3.4) we have

$$\widehat{y}_i = (a_i + \Delta a_i)^T x, \qquad |\Delta a_i| \le \gamma_n |a_i|.$$

This gives the backward error result

$$\widehat{y} = (A + \Delta A)x, \qquad |\Delta A| \le \gamma_n |A| \qquad (A \in \mathbb{R}^{m \times n}), \tag{3.10}$$

which implies the forward error bound

$$|y - \widehat{y}| \le \gamma_n |A||x|. \tag{3.11}$$

Normwise bounds readily follow (see Chapter 6 for norm definitions): for example,

$$\|y - \widehat{y}\|_p \le \gamma_n \|A\|_p \|x\|_p, \qquad p = 1, \infty.$$

This inner product formation of y can be expressed algorithmically as

```
% Sdot or inner product form.
y(1:m) = 0
for i = 1:m
    for j = 1:n
    end y(i) = y(i) + a(i,j)x(j)

end
```

The two loops can be interchanged to give

```
% Saxpy form.
y(1:m) = 0
for j = 1:n
    for i = 1:m
        y(i) = y(i) + a(i,j)x(j)
    end
end
```

The terms "sdot" and "saxpy" come from the BLAS (see §D.1). Sdot stands for (single precision) dot product, and saxpy for (single precision) a times x plus y. The question of interest is whether (3.10) and (3.11) hold for the saxpy form. They do: the saxpy algorithm still forms the inner products $a_i^T x$, but instead of forming each one in turn it evaluates them all "in parallel", a term at a time. The key observation is that exactly the same operations are performed, and hence *exactly the same rounding errors are committed-the* only difference is the order in which the rounding errors are created.

This "rounding error equivalence" of algorithms that are mathematically identical but algorithmically different is one that occurs frequently in matrix computations. The equivalence is not always as easy to see as it is for matrix-vector products.

Now consider matrix multiplication: $C = AB$, where $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$. Matrix multiplication is a triple loop procedure, with six possible loop orderings, one of which is

```
C(1:m,1:p) = 0
for i = 1:m
    for j = 1:p
        for k = 1:n
            C(i,j) = C(i,j) + A(i,k)B(k,j)
```

```
        end
    end
end
```

As for the matrix -vector product. all six versions commit the same rounding errors, so it suffices to consider any one of them. The "jik" and "jki" orderings both compute C a column at a time: $c_j = Ab_j$, where $c_j = C(:,j)$ and $b_j = B(:,j)$. From (3.10),

$$\widehat{c}_j = (A + \Delta A_j)b_j, \qquad |\Delta A_j| \le \gamma_n |A|.$$

Hence the jth computed column of C has a small backward error: it is the exact jth column for slightly perturbed data. The same cannot be said for $\widehat{C}$ as a whole (see Problem 3.5 for a possibly large backward error bound). However, we have the forward error bound

$$|C - \widehat{C}| \le \gamma_n |A||B| \qquad (A \in \mathbb{R}^{m\times n},\ B \in \mathbb{R}^{n\times p}), \tag{3.12}$$

and the corresponding normwise bounds include

$$\|C - \widehat{C}\|_p \le \gamma_n \|A\|_p \|B\|_p, \qquad p = 1, 2, F.$$

The bound (3.12) falls short of the ideal bound $|C - \widehat{C}| \le \gamma_n |C|$, which says that each component of C is computed with high relative accuracy. Nevertheless (3.12) is the best bound we can expect, because it reflects the sensitivity of the product to componentwise relative perturbations in the data: for any i and j we can find a perturbation ΔA with $|\Delta A| \le u|A|$ such that $|(A+\Delta A)B - AB|_{ij} = u(|A||B|)_{ij}$ (similarly for perturbations in B).

3.6. Complex Arithmetic

To carry out error analysis of algorithms in complex arithmetic we need a model for the basic arithmetic operations. Since complex arithmetic must be implemented using real arithmetic, the complex model is a consequence of the corresponding real one. We will assume that for complex numbers $x = a + ib$ and $y = c + id$ we compute

$$x \pm y = a + c \pm i(b + d), \tag{3.13a}$$

$$xy = ac - bd + i(ad + bc), \tag{3.13b}$$

$$x/y = \frac{ac + bd}{c^2 + d^2} + i\frac{bc - ad}{c^2 + d^2}. \tag{3.13c}$$

Lemma 3.5. *For $x, y \in \mathbb{C}$ the basic arithmetic operations computed according to* (3.13) *under the standard model* (2.4) *satisfy*

$$\begin{aligned} fl(x \pm y) &= (x+y)(1+\delta), & |\delta| &\le u, \\ fl(xy) &= xy(1+\delta), & |\delta| &\le \sqrt{2}\gamma_2, \\ fl(x/y) &= (x/y)(1+\delta), & |\delta| &\le \sqrt{2}\gamma_4. \end{aligned}$$

Proof. Throughout the proof, δ_i denotes a number bounded by $|\delta_i| \le u$. Addition/subtraction:

$$\begin{aligned} fl(x+y) &= (a+c)(1+\delta_1) + i(b+d)(1+\delta_2) \\ &= x + y + (a+c)\delta_1 + i(b+d)\delta_2, \end{aligned}$$

so

$$|fl(x+y) - (x+y)|^2 \le (|a+c|^2 + |b+d|^2)u^2 = (|x+y|u)^2,$$

as required.

Multiplication:

$$\begin{aligned} fl(xy) &= \big(ac(1+\delta_1) - bd(1+\delta_2)\big)(1+\delta_3) \\ &\quad + i\big(ad(1+\delta_4) + bc(1+\delta_5)\big)(1+\delta_6) \\ &= ac(1+\theta_2) - bd(1+\theta_2') + i\big(ad(1+\theta_2'') + bc(1+\theta_2''')\big) \\ &= xy + e, \end{aligned}$$

where

$$\begin{aligned} |e|^2 &\le \gamma_2^2\big((|ac| + |bd|)^2 + (|ad| + |bc|)^2\big) \\ &\le 2\gamma_2^2(a^2+b^2)(c^2+d^2) \\ &= 2\gamma_2^2|xy|^2, \end{aligned}$$

as required.

Division:

$$\begin{aligned} fl(c^2+d^2) &= \big(c^2(1+\delta_1) + d^2(1+\delta_2)\big)(1+\delta_3) \\ &= c^2(1+\theta_2) + d^2(1+\theta_2') \\ &= (c^2+d^2)(1+\theta_2''). \end{aligned}$$

Then

$$\begin{aligned} fl(\operatorname{Re} x/y) &= \frac{\big(ac(1+\delta_4) + bd(1+\delta_5)\big)(1+\delta_6)}{(c^2+d^2)(1+\theta_2'')} \\ &= \frac{ac(1+\theta_2''') + bd(1+\theta_2'''')}{(c^2+d^2)(1+\theta_2'')} \\ &= \operatorname{Re} x/y + e_1, \end{aligned}$$

where, using Lemma 3.3,

$$|e_1| \le \frac{|ac| + |bd|}{c^2 + d^2}\gamma_4.$$

Using the analogous formula for the error in $fl(\mathrm{Im}x/y)$,

$$\begin{aligned} - x/y|^2 &\le \frac{(|ac| + |bd|)^2 + (|bc| + |ad|)^2}{(c^2 + d^2)^2}\gamma_4^2 \\ &\le \frac{2(a^2 + b^2)(c^2 + d^2)}{(c^2 + d^2)^2}\gamma_4^2 \\ &= 2\gamma_4^2|x/y|^2, \end{aligned}$$

which completes the proof. □

It is worth stressing that δ in Lemma 3.5 is a complex number, so we cannot conclude from the lemma that the real and imaginary parts of $fl(x \text{ op } y)$ are obtained to high relative accuracy---only that they are obtained to high accuracy relative to $|x \text{ op } y|$.

As explained in §25.8, the formula (3.13c) is not recommended for practical use since it is susceptible to overflow. For the alternative formula (25.1), which avoids overflow, similar analysis to that in the proof of Lemma 3.5 shows that

$$fl(x/y) = (x/y)(1 + \delta), \qquad |\delta| \le \sqrt{2}\gamma_7.$$

Bounds for the rounding errors in the basic complex arithmetic operations are rarely given in the literature. Indeed, virtually all published error analyses in matrix computations are for real arithmetic. However, because the bounds of Lemma 3.5 are of the same form as for the standard model (2.4) for real arithmetic, most results for real arithmetic (including virtually all those in this book) are valid for complex arithmetic, provided that the constants are increased appropriately.

3.7. Miscellany

In this section we give some miscellaneous results that will be needed in later chapters. The first two results provide convenient ways to bound the effect of perturbations in a matrix product. The first result uses norms and the second. components.

Lemma 3.6. *If $X_j + \Delta X_j \in \mathbb{R}^{n\times n}$ satisfies $\|\Delta X_j\| \le d_j\|X_j\|$ for all j for a consistent norm, then*

$$\left\|\prod_{j=0}^{m}(X_j + \Delta X_j) - \prod_{j=0}^{m} X_j\right\| \le \left(\prod_{j=0}^{m}(1 + \delta_j) - 1\right)\prod_{j=0}^{m}\|X_j\|.$$

Proof. The proof is a straightforward induction, which we leave as an exercise (Problem 3.10). □

A componentwise result is entirely analogous.

Lemma 3.7. *If* $X_j + \Delta X_j \in \mathbb{R}^{n\times n}$ *satisfies* $|\Delta X_j| \le \delta_j |X_j|$ *for all j, then*

$$\left|\prod_{j=0}^{m}(X_j + \Delta X_j) - \prod_{j=0}^{m} X_j\right| \le \left(\prod_{j=0}^{m}(1+\delta_j) - 1\right)\prod_{j=0}^{m}|X_j|. \qquad \square$$

The final result describes the computation of the "rank 1 update" $y = (I - ab^T)x$, which is an operation arising in various algorithms, including the Gram-Schmidt method and Householder QR factorization.

Lemma 3.8. *Let* $a, b, x \in \mathbb{R}^n$ *and let* $y = (I - ab^T)x$ *be computed as* $\hat{y} = fl(x - a(b^T x))$. *Then* $\hat{y} = y + \Delta y$, *where*

$$|\Delta y| \le \gamma_{n+3}(I + |a||b^T|)|x|,$$

so that

$$\|\Delta y\|_2 \le \gamma_{n+3}(1 + \|a\|_2\|b\|_2)\|x\|_2.$$

Proof. Consider first the computation of $w = a(b^T x)$. We have

$$\begin{aligned}\hat{w} &:= (a + \Delta a)b^T(x + \Delta x), \qquad |\Delta a| \le u|a|, \quad |\Delta x| \le \gamma_n|x|,\\ &= a(b^T x) + a(b^T\Delta x) + \Delta a b^T(x + \Delta x)\\ &=: w + \Delta w,\end{aligned}$$

where

$$|\Delta w| \le \big(\gamma_n + u(1+\gamma_n)\big)|a||b^T||x|.$$

Finally, $\hat{y} = fl(x - \hat{w})$ satisfies

$$\hat{y} = x - a(b^T x) - \Delta w + \Delta y_1, \qquad |\Delta y_1| \le u(|x| + |\hat{w}|),$$

and

$$|-\Delta w + \Delta y_1| \le u(|x| + |a||b^T||x|) + (1+u)\big(\gamma_n + u(1+\gamma_n)\big)|a||b^T||x|.$$

Hence $\hat{y} = y + \Delta y$, where

$$\begin{aligned}|\Delta y| &\le \left[uI + (2u + u^2 + \gamma_n + 2u\gamma_n + u^2\gamma_n)|a||b^T|\right]|x|\\ &\le \gamma_{n+3}(I + |a||b^T|)|x|. \qquad \square\end{aligned}$$

3.8. Error Analysis Demystified

The principles underlying an error analysis can easily be obscured by the details. It is therefore instructive to examine the basic mechanism of forward and backward error analyses. We outline a general framework that reveals the essential simplicity.

Consider the problem of computing $z = f(a)$, where $f : \mathrm{IR}^n \to \mathrm{IR}^m$. Any algorithm for computing z can be expressed as follows. Let $x_1 = a$ and $x_{k+1} = g_k(x_k)$, $k = 1{:}p$, where

$$x_{k+1} = \begin{bmatrix} x_k \\ \xi_k \end{bmatrix}, \qquad \xi_k \in \mathbb{R}.$$

The kth stage of the algorithm represents a single floating point operation and x_k contains the original data together with all the intermediate quantities computed so far. Finally, $z = \bar{I} x_{p+1}$, where $\bar{I}$ is comprised of a subset of the columns of the identity matrix (so that each z_i is a component of x_{p+1}). In floating point arithmetic we have

$$\hat{x}_{k+1} = g_k(\hat{x}_k) + \Delta x_{k+1},$$

where Δ_{xk+i} represents the rounding errors on the kth stage and should be easy to bound. We assume that the functions g_k are continuously differentiable and denote the Jacobian of g_k at a by J_k. Then, to first order,

$$\begin{aligned}
\hat{x}_2 &= g_1(a) + \Delta x_2, \\
\hat{x}_3 &= g_2(\hat{x}_2) + \Delta x_3 = g_2(g_1(a) + \Delta x_2) + \Delta x_3 \\
&= g_2(g_1(a)) + J_2 \Delta x_2 + \Delta x_3, \\
\hat{x}_4 &= g_3(\hat{x}_3) + \Delta x_4 = g_3\big(g_2(g_1(a)) + J_2 \Delta x_2 + \Delta x_3\big) + \Delta x_4 \\
&= g_3\big(g_2(g_1(a))\big) + J_3 J_2 \Delta x_2 + J_3 \Delta x_3 + \Delta x_4.
\end{aligned}$$

The pattern is clear: for the final $\hat{z} = \hat{x}_{p+1}$ we have

$$\begin{aligned}
\hat{z} &= \bar{I}\big[g_p\big(\ldots g_2(g_1(a))\ldots\big) + J_p \ldots J_2 \Delta x_2 + J_p \ldots J_3 \Delta x_3 + \cdots \\
&\qquad + J_p \Delta x_p + \Delta x_{p+1}\big] \\
&= f(a) + \bar{I}\,\mathrm{diag}(J_p \ldots J_2, \ldots, J_p, I) \begin{bmatrix} \Delta x_2 \\ \Delta x_3 \\ \vdots \\ \Delta x_{p+1} \end{bmatrix} =: f(a) + Jh.
\end{aligned}$$

In a forward error analysis we bound $f(a) - \hat{z}$, which requires bounds for (products of) the Jacobians J_k. In a backward error analysis we write, again to first order,

$$f(a) + Jh = \hat{z} = f(a + \Delta a) = f(a) + J_f \Delta a,$$

where J_f is the Jacobian of f. So we need to solve, for Δa,

$$\underbrace{J_f}_{m\times n}\underbrace{\Delta a}_{n\times 1} = \underbrace{J}_{m\times q}\underbrace{h}_{q\times 1}, \qquad q = p(n + (p+1)/2).$$

In most matrix problems there are fewer outputs than inputs ($m < n$), so this is an underdetermined system. For a normwise backward error analysis we want a solution of minimum norm. For a componentwise backward error analysis, in which we may want (for example) to minimize ϵ subject to $|\Delta a| \le \epsilon|a|$, we can write

$$Jh = J_f D \cdot D^{-1}\Delta a =: Bc, \qquad D = \mathrm{diag}(a_i),$$

and then we want the solution c of minimal m-norm.

The conclusions are that forward error analysis corresponds to bounding derivatives and that backward error analysis corresponds to solving a large underdetermined linear system for a solution of minimal norm. In principal, therefore, error analysis is straightforward! Complicating factors in practice are that the Jacobians J_k may be difficult to obtain explicitly, that an error bound has to be expressed in a form that can easily be interpreted, and that we may want to keep track of higher-order terms.

3.9. Other Approaches

In this book we do not describe all possible approaches to error analysis. Some others are mentioned in this section.

Linearized rounding error bounds can be developed by applying equations that describe, to first order, the propagation of absolute or relative errors in the elementary operations +,-,*,/. The basics of this approach are given in many textbooks (see, for example, Dahlquist and Björck [262, 1974, §2.2] or Stoer and Bulirsch [955, 1980, §1.3]), but for a thorough treatment see Stummel [963, 1980], [964, 1981]. Ziv [1133, 1995] shows that linearized bounds can be turned into true bounds by increasing them by a factor that depends on the algorithm.

Rounding error analysis can be phrased in terms of graphs. This appears to have been first suggested by McCracken and Dorn [743, 1964], who use "process graphs" to represent a numerical computation and thereby to analyse the propagation of rounding errors. Subsequent more detailed treatments include those of Bauer [82, 1974], Miller [758, 1976], and Yalamov [1117, 1994]. The work on graphs falls under the heading of automatic error analysis (for more on which see Chapter 24) because processing of the large graphs required to represent practical computations is impractical by hand. Linnainmaa [705, 1976] shows how to compute the Taylor series expansion of the forward error

in an algorithm in terms of the individual rounding errors, and he presents a graph framework for the computation.

Some authors have taken a computational complexity approach to error analysis, by aiming to minimize the number of rounding error terms in a forward error bound, perhaps by rearranging a computation. Because this approach ignores the possibility of cancellation of rounding errors, the results need to be interpreted with care. See Aggarwal and Burgmeier [6, 1979] and Tsao [1024, 1983].

3.10. Notes and References

The use of Lemma 3.1 for rounding error analysis appears to originate with the original 1972 German edition of a book by Stoer and Bulirsch [955, 1980]. The lemma is also used, with $p_i = 1$, by Shampine and Allen [911, 1973, p. 18].

Lemma 3.4 is given by Forsythe and Moler [396, 1967, p. 92]. Wilkinson made frequent use of a slightly different version of Lemma 3.4 in which the assumption is $nu < 0.1$ and the bound for $|\eta_n|$ is $1.06nu$ (see, e.g., [1089, 1965, p. 113]).

A straight forward notation for rounding errors that is subsumed by the notation described in this chapter is suggested by Scherer and Zeller [899, 1980].

Ziv [1131, 1982] proposes the relative error measure

$$d(x, y) = \|x - y\| / \max(\|x\|, \|y\|)$$

for vectors x and y and explains some of its favourable properties for error analysis.

Wilkinson [1089, 1965, p. 447] gives error bounds for complex arithmetic; Olver [808, 1983] does the same in the relative precision framework. Demmel [280, 1984] gives error bounds that extend those in Lemma 3.5 by taking into account the possibility of underflow.

Henrici [521, 1980] gives a brief, easy to read introduction to the use of the model (2.4) for analysing the propagation of rounding errors in a general algorithm. He uses a set notation that is another possible notation to add to those in §3.4.

The perspective on error analysis in §3.8 was suggested by J. W. Demmel.

Problems

3.1. Prove Lemma 3.1.

3.2. (Kielbasiniski and Schwetlick [658, 1988], [659, 1992]) Show that if $p_i \equiv 1$ in Lemma 3.1 then the stronger bound $|\phi_n| \le nu/(1-\frac{1}{2}nu)$ holds for $nu < 2$.